

# Guía de Diseño v1.0

## Valdris: El Núcleo Profundo

Documento operativo de implementación · Complemento a la Guía de Proyecto v3

|           |                                                                        |
|-----------|------------------------------------------------------------------------|
| Proyecto  | Valdris: El Núcleo Profundo — Juego por Turnos JavaFX                  |
| Grupo     | H12GEXTRA — Marcos Castro · Ventura Pacheco · Marino Rodríguez         |
| Versión   | v1.0 — Punto 1: Sistema de turnos completo                             |
| Compañero | Guía de Proyecto v3.0 (narrativa, arquitectura, advertencias técnicas) |

### Propósito de este documento

- Documento operativo: se consulta mientras se programa, no solo al inicio del proyecto.
- Cada punto define con precisión cómo implementar un sistema concreto del juego.
- Se construye punto por punto — cada sección se añade tras definirla en detalle.
- Complementa la Guía de Proyecto v3 (historia, arquitectura, advertencias técnicas).
- Contenido actual: Punto 1 — Sistema de turnos. Se ampliará con los puntos 2 al 5.

---

## Punto 1 — Sistema de Turnos Completo

---

Esta sección define con precisión todo lo relativo al sistema de turnos del juego: qué puede hacer el jugador en su turno, en qué orden, qué restricciones aplican, cómo actúan los enemigos, y qué condiciones terminan la partida. Es la referencia definitiva para implementar TurnManager.

### 1.1 Acciones disponibles del jugador

En un turno el jugador puede realizar como máximo una acción de cada tipo, en el orden fijo definido a continuación. Ninguna acción es obligatoria — el jugador puede ceder el turno sin hacer nada.

| Acción      | Máximo por turno | Restricción                                                        | Notas                                                                                   |
|-------------|------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Movimiento  | 1                | Siempre va primero si se realiza                                   | BFS calcula celdas alcanzables. Consumir movePoints. Ver Sección 1.3.                   |
| Recogida    | 1                | Solo durante Fase 1 (item en suelo) o Fase 2 (contenedor contiguo) | Al pisar item en suelo: se ofrece automáticamente. Contenedor: acción manual en Fase 2. |
| Uso de item | 1                | Solo items del inventario. Usar arma = atacar (ver Ataque)         | Pociones, items especiales. No se puede usar y atacar con arma en el mismo turno.       |
| Ataque      | 1                | Solo si hay enemigo en rango del arma equipada                     | Rango depende del arma. Sin arma: rango 1 (cuerpo a cuerpo con daño base).              |
| Ceder turno | —                | En cualquier momento                                               | El jugador no hace nada. Pasa al turno enemigo.                                         |

#### Caso especial — usar arma vs atacar con arma

- Usar una espada del inventario = equiparla. Equipar NO es atacar.
- Atacar con la espada YA equipada SÍ es la acción de ataque.
- Flujo correcto en un turno: recoger espada (Fase 2) → está equipada → atacar con ella (Fase 4).
- El jugador PUEDE recoger una espada y atacar con ella en el mismo turno.
- El jugador NO PUEDE usar una poción Y atacar con arma en el mismo turno (son dos acciones distintas del mismo tipo).

### 1.2 Orden fijo de fases dentro del turno del jugador

El orden de las fases es fijo e inamovible. TurnManager avanza por fases secuencialmente. El jugador no puede hacer Fase 4 antes de Fase 1. Esto simplifica la implementación y evita situaciones ambiguas.

## Fase 1 — Movimiento

*El jugador elige moverse o no.*

- BFSMovimiento.getCellsInRange() calcula celdas alcanzables con movePoints disponibles.
- JavaFX resalta las celdas alcanzables. El jugador hace clic en su destino.
- Player.move(r,c) ejecuta el movimiento: descuenta movePoints, actualiza posición.
- Si la celda destino es DOOR o STAIRS: TurnManager.changeRoom() — ver Sección 1.4.
- Si la celda destino tiene item en suelo (requiresAdjacent=false): popup automático recoger/equipar/dejar.
- Si el jugador no se mueve: pasa directamente a Fase 2.

## Fase 2 — Recogida (opcional)

*Solo aplica si hay un contenedor (requiresAdjacent=true) en celda contigua.*

- TurnManager comprueba las 4 celdas adyacentes (N, S, E, O) a la posición actual del jugador.
  - Si hay celda con item.requiresAdjacent=true: se ofrece la opción de recoger.
  - El jugador elige: recoger y equipar, recoger y guardar en inventario, o ignorar.
  - Si no hay contenedor contiguo: fase se salta automáticamente.
  - La recogida de items en suelo ya ocurrió en Fase 1 al pisar la celda.
- *Diferencia clave: item en suelo = se pisa y se ofrece (Fase 1). Contenedor = está en celda adyacente y se recoge activamente (Fase 2).*

## Fase 3 — Uso de item (opcional)

*El jugador puede usar un item de su inventario.*

- El jugador selecciona un item del inventario en el panel lateral de JavaFX.
- Item.use(player) ejecuta el efecto: curación, buff, efecto especial.
- Usar un arma en esta fase = equiparla. No cuenta como ataque.
- Solo se puede usar 1 item por turno en esta fase.
- Si el jugador no quiere usar nada: pasa a Fase 4.

## Fase 4 — Ataque (opcional)

*El jugador puede atacar un enemigo en rango.*

- El jugador selecciona un enemigo en rango (resaltado en JavaFX).
- `Player.attack(enemy)`: calcula daño = daño base + arma equipada - defensa enemigo.
- Si el enemigo muere: `Enemy.onDeath()` → `dropltem` si tiene, se elimina de la Cola de turnos.
- Solo se puede atacar 1 vez por turno.
- Si no hay enemigos en rango o el jugador no quiere atacar: termina el turno del jugador.

## 1.3 Reglas de movimiento y walkability

Una celda es alcanzable por el BFS solo si cumple ambas condiciones simultáneamente. Estas condiciones se encapsulan en `Cell.isWalkable()` para que el BFS no tenga que conocer los detalles internos de cada tipo de celda.

| Condición                | Celdas transitables                                                                                                    | Celdas bloqueadas                                                                                       |
|--------------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Tipo de celda (CellType) | FLOOR, DOOR, DOOR_HIDDEN (si activada), RUNE, LEVER, STAIRS_UP, STAIRS_DOWN                                            | WALL, DOOR_LOCKED (sin llave)                                                                           |
| Contenido de la celda    | <code>cell.unit == null</code> Y ( <code>cell.item == null</code> O <code>cell.item.requiresAdjacent == false</code> ) | <code>cell.unit != null</code> (enemigo) O <code>cell.item.requiresAdjacent == true</code> (contenedor) |

```
// Cell.java
public boolean isWalkable() {
    if (type == CellType.WALL) return false;
    if (type == CellType.DOOR_LOCKED) return false;
    if (unit != null) return false;
    if (item != null && item.isRequiresAdjacent()) return false;
    return true;
}
```

## Items en suelo vs contenedores — resumen

- Item en suelo (`requiresAdjacent=false`): la celda ES transitable. El jugador la pisa y se ofrece recogerlo.
- Contenedor (`requiresAdjacent=true`): la celda NO es transitable. El jugador interactúa desde celda adyacente en Fase 2.
- Enemigo vivo: la celda NO es transitable. El jugador ataca desde rango, no se mueve encima.
- Enemigo muerto: si soltó item en suelo, la celda SÍ es transitable (el item ya no es un contenedor).

## 1.4 Cambio de habitación y escaleras

Cuando el jugador se mueve a una celda DOOR o STAIRS, `TurnManager` gestiona el cambio de sala. Los enemigos NO siguen al jugador entre salas — cada Room tiene su propia lista de enemigos activos.

| Tipo de celda           | Qué ocurre                                                                                                                                                              | Condición previa                                    |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| DOOR                    | Dungeon.moveToAdjacentRoom(direction): actualiza currentRoom. TurnManager carga enemigos de la nueva sala en la Cola. Se llama a onRoomEnter() para diálogos y eventos. | Ninguna — la puerta está abierta.                   |
| DOOR_LOCKED             | TurnManager comprueba Player.inventory. Si tiene la llave correcta: Cell.type = DOOR y se procesa como DOOR. Si no: mensaje de bloqueo, el movimiento no ocurre.        | Player debe tener el item llave en inventario.      |
| DOOR_HIDDEN             | La celda aparece como FLOOR hasta que Room.checkSecretTrigger() la activa. Una vez activada se trata como DOOR normal.                                                  | El trigger de activación debe haberse pisado antes. |
| STAIRS_UP / STAIRS_DOWN | Dungeon.changeFloor(direction): cambia floorLevel. Carga el grafo del nuevo piso. Se actualiza currentRoom al nodo de entrada del nuevo piso.                           | Ninguna — las escaleras siempre son accesibles.     |

## 1.5 Turno de enemigos y defensa automática

Tras completar el turno del jugador, TurnManager procesa los enemigos de la sala actual en el orden en que están en la Cola. Solo actúan los enemigos de la sala actual — los enemigos de otras salas no se procesan.

| Paso                              | Qué hace TurnManager                                                                                                                                                 |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Filtrar enemigos activos       | Iterar Cola y seleccionar solo los Enemy cuyo roomId == currentRoom.getId() y cuyo hp > 0.                                                                           |
| 2. Ejecutar turno de cada enemigo | Para cada Enemy activo: llamar a enemy.executeTurn(player, room). El enemigo decide mover y/o atacar según su IA (ver Punto 2).                                      |
| 3. Defensa automática del jugador | Cuando un Enemy ataca al Player: TurnManager calcula daño = enemy.damage - player.getDefense(). Player.takeDamage(daño). No requiere acción del jugador.             |
| 4. MalacharAlly (solo Zona 5)     | Si MalacharAlly está en la Cola: executeAllyTurn(player, room). Actúa después del jugador y antes de los enemigos. Sus acciones se muestran diferenciadas en el log. |
| 5. Actualizar log                 | TurnManager.addLog() registra cada acción en LSE. Formato: '[Turno X] Warrior ataca a Kael por 8 de daño.'                                                           |

## 1.6 Contadores de turno y condiciones de fin de partida

El juego tiene dos contadores de turno independientes que corren en paralelo. Ambos se decrementan al final de cada turno completo (jugador + enemigos).

| Contador                   | Dónde se guarda                  | Cuándo decrementa                         | Condición de derrota                                                                |
|----------------------------|----------------------------------|-------------------------------------------|-------------------------------------------------------------------------------------|
| Turnos globales de partida | TurnManager.globalTurnsLeft: int | Siempre, al final de cada turno completo. | globalTurnsLeft == 0 → derrota inmediata, independientemente del estado de la sala. |

| Contador       | Dónde se guarda         | Cuándo decremanta                                                                  | Condición de derrota                                        |
|----------------|-------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Turnos de sala | Room.roomTurnsLeft: int | Solo si currentRoom.hasRoomTimer == true. Las salas de exploración NO decrementan. | roomTurnsLeft == 0 → derrota aunque queden turnos globales. |

### Salas de exploración vs salas con temporizador

- Room.hasRoomTimer = false → sala de exploración. El jugador puede explorar sin presión de tiempo.
- Room.hasRoomTimer = true → sala con temporizador. roomTurnsLeft decremanta cada turno.
- Algunas salas de exploración pueden tener un evento activador: tras X turnos, hasRoomTimer se activa automáticamente (enemigo que despierta, trampa que se arma...).
- El contador de sala se reinicia al salir y volver a entrar a la misma sala — para evitar abuso de reseteo, se puede marcar Room.timerConsumed = true tras agotarse.

| Condición                             | Resultado                     | Quién lo detecta                                     |
|---------------------------------------|-------------------------------|------------------------------------------------------|
| HP del jugador <= 0                   | Derrota                       | TurnManager tras recibir daño                        |
| globalTurnsLeft == 0                  | Derrota                       | TurnManager al final de cada turno                   |
| roomTurnsLeft == 0                    | Derrota                       | TurnManager al final de cada turno (si hasRoomTimer) |
| Todos los enemigos de la sala muertos | Sala despejada — avanzar      | TurnManager tras Enemy.onDeath()                     |
| ParasitoEnemy muerto (Zona 5)         | Victoria — secuencia de final | TurnManager.triggerEnding(characterType)             |

## 1.7 Estructura de TurnManager

TurnManager es la clase central que orquesta todo el sistema de turnos. No contiene lógica de negocio del juego — delega en Player, Enemy y Room. Su responsabilidad es el flujo, el orden y el log.

```

public class TurnManager {
    private Cola turnQueue; // orden de actuación
    private LSE log; // historial de acciones
    private GameState gameState; // estado serializable
    private int globalTurnsLeft; // contador global
    private TurnPhase currentPhase; // fase actual del turno
    // Fases del turno del jugador
    public enum TurnPhase {
        PLAYER_MOVE, PLAYER_PICKUP, PLAYER_USE_ITEM, PLAYER_ATTACK,
        ENEMY_TURNS, CHECK_CONDITIONS, TURN_END
    }
    public void nextTurn() { ... } // avanza una fase
    public void addLog(String msg) { ... } // añade al historial
    public void onRoomEnter() { ... } // diálogos y eventos de sala
    public void changeRoom(String dir) { ... }
    public void triggerEnding(CharacterType t) { ... }

```

```
public void reconstruirReferencias() { ... } // tras cargar JSON
}
```

## 1.8 División de trabajo recomendada

| Miembro     | Bloque                        | Qué incluye                                                                                                              |
|-------------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Marino (tú) | Motor del juego + integración | Modelo completo (Fase 1), lógica del juego (Fase 2: BFS, TurnManager, combate, acertijos), integración final con JavaFX. |
| Compañero 2 | Persistencia + tests          | Fase 3 (LectorJSON, GameState, serialización plana). Tests JUnit para todas las clases del modelo desde el día 1.        |
| Compañero 3 | Interfaz JavaFX               | Fase 4: GridPane, paneles, selección de personaje, diálogos, pantallas de final. Empieza en día 10.                      |

### Regla de oro para el compañero de JavaFX

- Los Controllers de JavaFX NO pueden contener lógica de juego.
- Un Controller solo llama a métodos de TurnManager o Player — nunca calcula daño, nunca mueve unidades directamente.
- Antes de que empiece: sesión de 30 minutos explicando qué puede y qué no puede hacer un Controller.
- Si hay duda: si el método devuelve un int o modifica un atributo de un modelo, no va en el Controller.